BeaconKit 아키텍처 분석 - 02. 서비스 레지스트리 및 생명주기 관리

1. 서비스 레지스트리 개요

BeaconKit은 **Service Registry 패턴**을 사용하여 모든 서비스의 생명주기를 중앙에서 관리합니다. 이는 의존성 관리를 용이하게 하고 메모리에서 동일한 참조를 사용하도록 보장합니다.

1.1 핵심 인터페이스

```
// Basic은 서비스의 최소 인터페이스입니다
type Basic interface {
// Start는 서비스에 필요한 고루틴을 시작합니다
Start(ctx context.Context) error
// Stop은 서비스를 중지합니다. 시작되지 않은 서비스에서도 안전하게 호출 가능
Stop() error
// Name은 서비스의 이름을 반환합니다
Name() string
}
```

1.2 Registry 구조

```
type Registry struct {
```

```
// logger는 Registry의 로거입니다

logger log.Logger

// services는 서비스 타입 -> 서비스 인스턴스 맵입니다

services  map[string]Basic

// servicesStarted는 Start()가 호출된 서비스들의 맵입니다

servicesStarted  map[string]struct{}

// serviceTypes는 등록된 서비스 타입들의 순서가 있는 슬라이스입니다

serviceTypes []string

}
```

2. 서비스 시작 순서

BeaconKit의 서비스들은 **의존성에 따라 엄격한 순서**로 시작됩니다. 이 순서는 `ProvideServiceRegistry` 함수에서 정의됩니다:

2.1 시작 순서 (1 → 8)

```
opts := []service.RegistryOption{

// 1. ShutdownService - 첫 번째로 시작, 마지막에 중지

service.WithService(in.ShutdownService),

// 2. ValidatorService - 블록 빌딩 서비스

service.WithService(in.ValidatorService),

// 3. NodeAPIServer - REST API 서버

service.WithService(in.NodeAPIServer),
```

```
// 4. ReportingService - 버전 리포팅
service.WithService(in.ReportingService),
// 5. TelemetryService - 메트릭 수집
service.WithService(in.TelemetryService),
// 6. EngineClient - 실행 레이어 연결 (블로킹)
service.WithService(in.EngineClient),
// 7. ChainService - 블록체인 서비스
service.WithService(in.ChainService),
// 8. CometBFTService - 합의 엔진 (마지막)
service.WithService(in.CometBFTService),
}
```

2.2 중지 순서 (8 → 1)

서비스 중지는 **시작의 역순**으로 진행됩니다:

```
func (s *Registry) StopAll() {
// 시작 순서의 역순으로 서비스 중지
for i := len(s.serviceTypes) - 1; i >= 0; i-- {
typeName := s.serviceTypes[i]
// ... 중지 로직
}
}
```

3. 각 서비스의 역할과 의존성

3.1 ShutdownService

- **역할**: 시스템 종료 신호 처리
- **의존성**: 없음 (독립적)
- **특징**: 첫 번째로 시작, 마지막에 중지

3.2 ValidatorService

- **역할**: 블록 빌딩 및 검증
- **의존성**: StateProcessor, BlobFactory, PayloadBuilder
- **핵심 기능**:

```
type Service struct {
cfg *Config
logger log.Logger
chainSpec ChainSpec
signer crypto.BLSSigner
blobFactory BlobFactory
sb StorageBackend
stateProcessor StateProcessor
localPayloadBuilder PayloadBuilder
metrics *validatorMetrics
}
```

3.3 NodeAPIServer

- **역할**: REST API 엔드포인트 제공
- **의존성**: StorageBackend (읽기 전용)
- **특징**: 합의 흐름과 독립적으로 동작

3.4 ReportingService

- **역할**: 노드 버전 및 상태 리포팅
- **의존성**: 최소한의 시스템 정보
- **특징**: 백그라운드 모니터링

3.5 TelemetryService

- **역할**: 메트릭 수집 및 전송
- **의존성**: Config
- **특징**: 성능 모니터링 및 관찰 가능성

3.6 EngineClient (중요한 블로킹 포인트)

- **역할**: 실행 레이어 클라이언트와의 연결
- **의존성**: JWT 시크릿, RPC URL
- **특징**: **실행 레이어 연결까지 블로킹**
- **중요성**: 이후 서비스들이 실행 엔진에 의존하므로 연결 필수

3.7 ChainService (BlockchainService)

- **역할**: 블록 처리 및 상태 전환
- **의존성**: ExecutionEngine, StateProcessor, StorageBackend

- **핵심 구조**:

```
type Service struct {
storageBackend StorageBackend
blobProcessor BlobProcessor
depositContract deposit.Contract
executionEngine ExecutionEngine
localBuilder LocalBuilder
stateProcessor StateProcessor
// ... 기타 필드들
}
```

3.8 CometBFTService (마지막)

- **역할**: 합의 엔진 및 P2P 네트워킹
- **의존성**: ChainService (ABCI 앱으로 사용)
- **특징**: 전체 시스템의 드라이버 역할

4. 생명주기 관리 메커니즘

4.1 서비스 등록

```
func (s *Registry) RegisterService(service Basic) error {
typeName := service.Name()
if _, exists := s.services[typeName]; exists {
```

```
return errServiceAlreadyExists
}
s.services[typeName] = service
s.serviceTypes = append(s.serviceTypes, typeName)
return nil
}
```

4.2 서비스 시작

```
func (s *Registry) StartAll(ctx context.Context) error {
s.logger.Info("Starting services", "num", len(s.serviceTypes))
for _, typeName := range s.serviceTypes {
s.logger.Info("Starting service", "type", typeName)
svc := s.services[typeName]
if err := svc.Start(ctx); err != nil {
return fmt.Errorf("error when starting service %s: %w", typeName, err)
}
s.servicesStarted[typeName] = struct{}{}
}
return nil
}
```

4.3 서비스 조회

```go
func (s *Registry) FetchService(service  interface{}) error {
// 리플렉션을 사용하여 타입 매칭
serviceType := reflect.TypeOf(service)
// ... 타입 검증 및 할당 로직
}
```

5. 의존성 주입 (Dependency Injection)

BeaconKit은 `cosmossdk.io/depinject` 를 사용하여 의존성 주입을 구현합니다.

5.1 컴포넌트 제공자 패턴

각 컴포넌트는 `Provide*` 함수를 통해 제공됩니다:

```go
// ValidatorServiceInput은 validator service provider의 입력입니다
type ValidatorServiceInput struct {
depinject.In
Cfg *Config
Logger *phuslu.Logger
ChainSpec chain.ChainSpec
StorageBackend StorageBackend
StateProcessor StateProcessor
Signer crypto.BLSSigner
BlobFactory BlobFactory
```

```
LocalPayloadBuilder PayloadBuilder

TelemetrySink TelemetrySink

}


// ProvideValidatorService는 validator service의 depinject provider입니다

func  ProvideValidatorService(in ValidatorServiceInput) (*validator.Service, err

return  validator.NewService(

in.Cfg,

in.Logger,

in.ChainSpec,

in.StorageBackend,

in.StateProcessor,

in.Signer,

in.BlobFactory,

in.LocalPayloadBuilder,

in.TelemetrySink,

), nil

}
```

5.2 노드 빌더에서의 조립

```
func (nb *NodeBuilder) Build(...) types.Node {

var (
```

```
beaconNode types.Node

cmtService types.ConsensusService

config *config.Config

)



// depinject를 사용하여 모든 노드 컴포넌트 빌드

if  err = depinject.Inject(

depinject.Configs(

depinject.Provide(

nb.components..., // 모든 Provide* 함수들

),

depinject.Supply(

appOpts,

logger,

db,

cmtCfg,

chainSpec,

),

),

&beaconNode,

&cmtService,

&config,

);  err != nil {
```

```
panic(err)
}
return beaconNode
}
```

6. 오류 처리 및 복구

6.1 시작 실패 처리

```
if err := svc.Start(ctx); err != nil {
return fmt.Errorf("error when starting service %s: %w", typeName, err)
}
```

- 하나의 서비스 시작 실패 시 전체 시스템 시작 중단
- 이미 시작된 서비스들은 자동으로 중지되어야 함

6.2 중지 실패 처리

```
if err := svc.Stop(); err != nil {
s.logger.Error("error when stopping service", "type", typeName, "err", err)
}
```

- 중지 실패는 로그만 남기고 계속 진행
- 다른 서비스들의 중지를 방해하지 않음

7. 중요한 블로킹 포인트들

7.1 EngineClient 연결 대기

EngineClient는 실행 레이어와의 연결이 성공할 때까지 **블로킹**됩니다:

- JWT 인증 설정
- RPC 연결 확인
- 실행 클라이언트 응답성 검증

7.2 서비스 간 동기화

- **ValidatorService**와 **ChainService**는 모두 **ExecutionEngine**에 의존
- **CometBFTService**는 **ChainService**를 ABCI 앱으로 사용
- 순서가 중요한 이유: 의존성 체인이 올바르게 구성되어야 함

8. 모니터링 및 관찰 가능성

8.1 서비스 상태 추적

```
type Registry struct {
servicesStarted  map[string]struct{} // 시작된 서비스 추적
}
```

8.2 로깅

- 각 서비스 시작/중지 시 로그 출력
- 오류 발생 시 상세한 컨텍스트 제공

- 전체 서비스 수 및 상태 요약

9. 설계 원칙

9.1 단일 책임 원칙

- 각 서비스는 명확한 단일 책임을 가짐
- Registry는 오직 생명주기 관리만 담당

9.2 의존성 역전 원칙

- 인터페이스를 통한 느슨한 결합
- DI 컨테이너를 통한 의존성 주입

9.3 실패 안전성

- 서비스 시작 실패 시 전체 시스템 중단
- 서비스 중지 실패는 다른 서비스에 영향 없음

이 서비스 레지스트리 시스템은 BeaconKit의 복잡한 컴포넌트들을 체계적으로 관리하며, 안정적인 시작/중지 순서를 보장합니다. 다음 문서에서는 CometBFT와의 통합 및 ABCI 인터페이스에 대해 자세히 살펴보겠습니다.